UNIVERSIDADE FEDERAL DE SERGIPE

Geração Combinatória

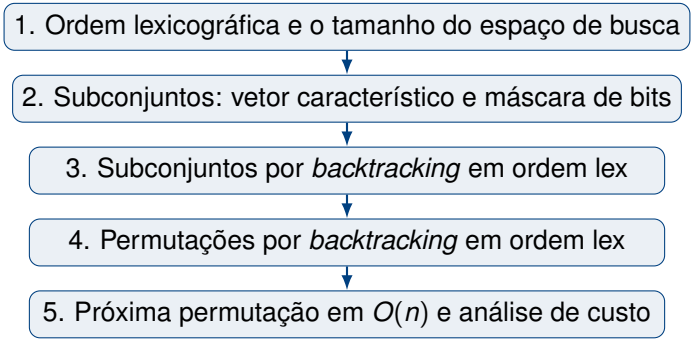
Subconjuntos e permutações em ordem lexicográfica

Prof. Dr. André Yoshiaki Kashiwabara

Algoritmos e Estruturas de Dados II — DCOMP/UFS

4 de novembro de 2026

O que você vai aprender hoje



Ordem lexicográfica e espaço de busca

Por que gerar todas as configurações?

Problema

Muitos problemas não têm algoritmo eficiente conhecido. A alternativa é *examinar todas as configurações candidatas*:

- subconjuntos: soma de subconjunto, mochila exata, cobertura;
- permutações: caixeiro viajante.

Solução

Um *gerador* que percorre o espaço de busca:

- sem repetir nenhuma configuração;
- sem esquecer nenhuma;
- em uma ordem *previsível* — a ordem lexicográfica.

Ordem lexicográfica: definição

Definição

Sejam $a = (a_1, \ldots, a_p)$ e $b = (b_1, \ldots, b_q)$ sequências sobre um conjunto ordenado. Dizemos que $a \prec_{\text{lex}} b$ quando:

- existe $i \leq \min(p, q)$ com $a_j = b_j$ para todo $j < i$ e $a_i < b_i$; **ou**
- $p < q$ e a é prefixo de b.

Intuição

É a ordem do dicionário: compara-se posição a posição e, na primeira diferença, decide-se. Quem acaba antes e é prefixo do outro vem primeiro.

Onde aparece: Knuth [2], seç. 7.2.1 (ordens de geração); Kreher & Stinson [3], cap. 2.

Ordem lexicográfica: exemplos

Comparações

- $(1,3) \prec_{\text{lex}} (2)$ pois $1 < 2$
- $(1,2) \prec_{\text{lex}} (1,3)$ primeira diferença na 2ª posição
- $(1) \prec_{\text{lex}} (1,2)$ prefixo vem antes
- $(2,3) \prec_{\text{lex}} (3)$ pois $2 < 3$

Os 8 subconjuntos de $\{1,2,3\}$ em ordem lex

$$\emptyset \prec \{1\} \prec \{1,2\} \prec \{1,2,3\} \prec \{1,3\} \prec \{2\} \prec \{2,3\} \prec \{3\}$$

As 6 permutações de $\{1,2,3\}$ em ordem lex

$$123 \prec 132 \prec 213 \prec 231 \prec 312 \prec 321$$

O tamanho do espaço de busca

n	2^n	$n!$
5	32	120
10	1 024	$3{,}6 \times 10^6$
15	32 768	$1{,}3 \times 10^{12}$
20	$1{,}0 \times 10^6$	$2{,}4 \times 10^{18}$

Consequência prática

Subconjuntos são viáveis até $n \approx 25$–30; permutações, até $n \approx 11$–13.

Qualquer trabalho *extra* por configuração multiplica um número já enorme: o gerador precisa ser econômico.

Onde aparece: Cormen et al. [1], Apêndice C.1 (contagem: permutações e combinações); Knuth [2], seç. 7.2.1.

Recapitulando: ordem lexicográfica

O que aprendemos

- Enumeração exaustiva é a estratégia-base para problemas sem algoritmo eficiente.
- A ordem lexicográfica compara sequências posição a posição; prefixo vem antes.
- Gerar em ordem lex dá saída determinística, fácil de conferir e de comparar.
- 2^n subconjuntos e $n!$ permutações limitam severamente o n tratável.

Subconjuntos: vetor característico

Por que representar subconjuntos por bits?

Problema

Um subconjunto é um objeto de tamanho variável. Como percorrer *todos* eles sem estrutura de dados complicada?

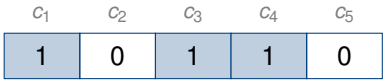
Solução

Cada subconjunto vira um vetor de n bits — e portanto um inteiro entre 0 e $2^n - 1$. Enumerar subconjuntos vira *contar*.

Vetor característico

Definição

Para $S \subseteq \{1, \ldots, n\}$, o vetor característico é $c(S) = (c_1, \ldots, c_n)$ com $c_i = 1$ se $i \in S$ e $c_i = 0$ caso contrário. A correspondência $S \leftrightarrow c(S)$ é uma bijeção entre os 2^n subconjuntos e os 2^n vetores binários de tamanho n.



$\iff S = \{1, 3, 4\}$

$\iff 10110_2 = 22$

Enumerando por contagem binária

```c
/* Percorre os 2^n subconjuntos de {1, ..., n}.
   Bit mais significativo (posicao n-1) representa o elemento 1. */
void subconjuntos_mascara(int n)
{
    for (unsigned m = 0; m < (1u << n); m++) {
        printf("{");
        for (int i = 1; i <= n; i++)
            if (m & (1u << (n - i)))   /* elemento i pertence a S? */
                printf(" %d", i);
        printf(" }\n");
    }
}
```

- Custo: $\Theta(n \cdot 2^n)$ — n testes de bit por subconjunto.
- Espaço adicional: $O(1)$. Limite prático: $n \leq 31$ com `unsigned`.

Onde aparece: Knuth [2], seç. 7.2.1.1 (geração de todas as n-uplas); Kreher & Stinson [3], seç. 2.2 (subsets).

Cuidado: duas ordens diferentes

Contagem binária ($m = 0, 1, \ldots, 7$)

m	S
000	$\emptyset$
001	$\{3\}$
010	$\{2\}$
011	$\{2,3\}$
100	$\{1\}$
101	$\{1,3\}$
110	$\{1,2\}$
111	$\{1,2,3\}$

Ordem lex dos *elementos*

$\emptyset$, $\{1\}$, $\{1,2\}$, $\{1,2,3\}$,
$\{1,3\}$, $\{2\}$, $\{2,3\}$, $\{3\}$

Observação

A contagem binária é lex sobre os *vetores de bits*, não sobre as *listas crescentes de elementos*. Para obter a segunda ordem usamos *backtracking*.

Recapitulando: vetor característico

O que aprendemos

- Subconjunto $\leftrightarrow$ vetor de n bits $\leftrightarrow$ inteiro em $[0, 2^n)$.
- Enumerar todos os subconjuntos é contar de 0 a $2^n - 1$: código curto, $O(1)$ de espaço.
- Custo $\Theta(n \cdot 2^n)$; cabe em `unsigned` até $n = 31$.
- A ordem produzida é lex *nos bits* — diferente da ordem lex *nos elementos*.

Subconjuntos por backtracking

Por que backtracking para subconjuntos?

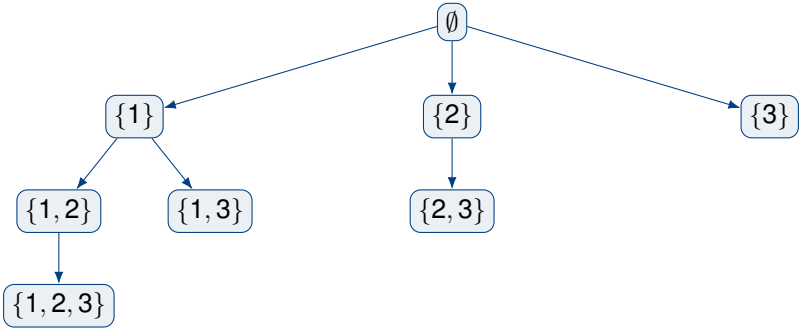
Problema

Queremos a ordem lex sobre as listas crescentes de elementos — e queremos poder *podar* ramos inteiros quando o prefixo já é inviável (soma estourou, restrição violada). A máscara de bits não permite isso.

Solução

Construir o subconjunto incrementalmente: em cada nível, escolher o *próximo* elemento entre os maiores que o último escolhido, em ordem crescente. Visitar o nó ao chegar nele.

A árvore de enumeração



Percurso em pré-ordem, filhos da esquerda para a direita $\Rightarrow$ ordem lexicográfica.

Algoritmo em C

```c
int n;            /* universo {1, ..., n}  */
int sol[MAXN];    /* prefixo em construcao */
int k;            /* tamanho do prefixo    */

void subconjuntos(int inicio)
{
    visita(sol, k);                      /* 1. visita o no atual */
    for (int i = inicio; i <= n; i++) {  /* 2. filhos em ordem crescente */
        sol[k++] = i;                    /*    escolhe i */
        subconjuntos(i + 1);             /*    so elementos maiores que i */
        k--;                             /*    desfaz a escolha */
    }
}
/* chamada inicial: k = 0; subconjuntos(1); */
```

- `inicio = i + 1` garante subconjunto *crescente* e sem repetição.
- Profundidade máxima da recursão: n; espaço $O(n)$.

Onde aparece: Skiena [5], seç. 9.2.1; Kreher & Stinson [3], cap. 4; Ziviani [6], seç. 2.3.

Por que a saída sai em ordem lex?

Argumento

Em cada nó, o prefixo já fixado é comum a toda a subárvore. Os filhos são gerados com valores *i crescentes*, logo a subárvore do filho i contém apenas sequências menores (lex) que as da subárvore do filho $i+1$. Como o nó é visitado *antes* dos filhos, o prefixo aparece antes de suas extensões — exatamente a segunda cláusula da definição de $\prec_{\text{lex}}$.

Exemplo: $n = 3$

Saída: $\emptyset,\ \{1\},\ \{1,2\},\ \{1,2,3\},\ \{1,3\},\ \{2\},\ \{2,3\},\ \{3\}$.

Onde a poda entra

Soma de subconjunto

Queremos os S com $\sum_{i \in S} w_i = A$, com todos os $w_i > 0$.

```
1  void busca(int inicio, int soma)
2  {
3      if (soma == A) visita(sol, k);
4      if (soma >= A) return;                /* poda: pesos positivos */
5      for (int i = inicio; i <= n; i++) {
6          sol[k++] = i;
7          busca(i + 1, soma + w[i]);
8          k--;
9      }
10 }
```

- Uma linha corta subárvores inteiras — impossível na enumeração por máscara.
- A ordem de visita dos ramos sobreviventes continua lexicográfica.

Onde aparece: Skiena [5], seç. 9.3 (*search pruning*); Kreher & Stinson [3], seç. 4.6; Cormen et al. [1], seç. 35.5.

Recapitulando: subconjuntos por backtracking

O que aprendemos

- Escolher o próximo elemento sempre *maior* que o último evita repetições.
- Pré-ordem com filhos crescentes $\Rightarrow$ ordem lexicográfica dos elementos.
- Espaço $O(n)$; tempo $\Theta(n \cdot 2^n)$ no pior caso, muito menos com poda.
- A poda é a vantagem decisiva sobre a máscara de bits.

Permutações por backtracking

Por que permutações são diferentes

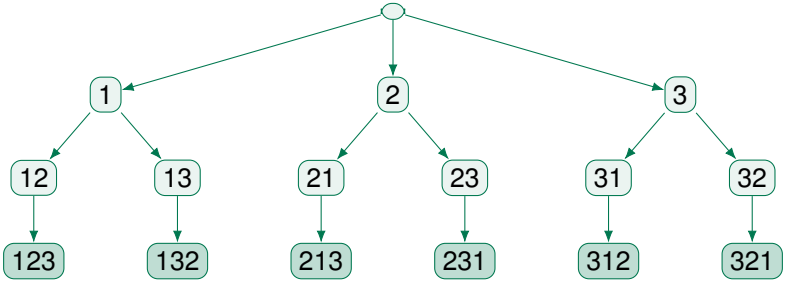
Problema

Em um subconjunto, cada elemento entra ou não. Em uma permutação, todos entram — o que muda é a *posição*. Precisamos saber quais valores ainda estão disponíveis.

Solução

Preencher as posições $0, 1, \ldots, n-1$ e, em cada posição, testar os valores livres em ordem crescente, marcando-os em um vetor `usado[]`.

A árvore de permutações de $\{1, 2, 3\}$



Só as folhas são visitadas: $n!$ folhas, todas na profundidade n.

Algoritmo em C

```c
int n;
int sol[MAXN];           /* permutacao parcial          */
int usado[MAXN + 1];     /* usado[v] = 1 se v ja esta em sol */

void permutacoes(int pos)
{
    if (pos == n) { visita(sol, n); return; }   /* folha */
    for (int v = 1; v <= n; v++) {              /* valores em ordem crescente */
        if (usado[v]) continue;
        usado[v] = 1;  sol[pos] = v;
        permutacoes(pos + 1);
        usado[v] = 0;                           /* desfaz */
    }
}
/* chamada inicial: memset(usado, 0, sizeof usado); permutacoes(0); */
```

- Testar v em ordem crescente em cada posição $\Rightarrow$ folhas em ordem lex.
- Tempo $\Theta(n \cdot n!)$; espaço $O(n)$.

Onde aparece: Skiena [5], seç. 9.2.2 (*constructing all permutations*); Kreher & Stinson [3], seç. 2.4; Ziviani [6], seç. 2.3.

Contraste: geração por trocas

```c
void permuta_troca(int v[], int pos, int n)   /* NAO gera em ordem lex */
{
    if (pos == n) { visita(v, n); return; }
    for (int i = pos; i < n; i++) {
        troca(&v[pos], &v[i]);
        permuta_troca(v, pos + 1, n);
        troca(&v[pos], &v[i]);
    }
}
```

Saída para 123

123, 132, 213, 231, 321, 312

Dispensa `usado[]` e é mais rápida por constante, mas a troca *desordena* o sufixo. Use-a quando a ordem não importa.

Onde aparece: Sedgewick [4] (levantamento dos métodos por troca); Knuth [2], seç. 7.2.1.2, Algoritmo P (*plain changes*).

Recapitulando: permutações por backtracking

O que aprendemos

- Preencher posição a posição, testando valores livres em ordem crescente.
- `usado[]` custa $O(n)$ de espaço e mantém a ordem lexicográfica.
- Apenas folhas são visitadas: $n!$ saídas, custo $\Theta(n \cdot n!)$.
- A variante por trocas é mais enxuta, mas não respeita a ordem lex.

Próxima permutação e análise de custo

Por que um sucessor iterativo?

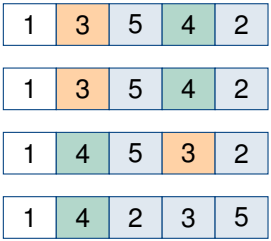
Problema

A recursão obriga a percorrer tudo de uma vez. Às vezes queremos *avançar uma permutação por vez*: dentro de um laço, retomando de onde parou, sem pilha de recursão.

Solução

Dado um arranjo, calcular *in loco* a menor permutação estritamente maior que ele — o sucessor lexicográfico, em $O(n)$. É o que faz `std::next_permutation`.

Os quatro passos



1. sufixo decrescente $(5, 4, 2)$; pivô $= 3$ (último $v_i < v_{i+1}$)
2. mais à direita maior que o pivô: 4
3. troca pivô com ele: 1 4 5 3 2
4. inverte o sufixo: 1 4 2 3 5 é o sucessor

Implementação em C

```c
static void troca(int *a, int *b) { int t = *a; *a = *b; *b = t; }

static void inverte(int v[], int i, int j)
{
    while (i < j) { troca(&v[i], &v[j]); i++; j--; }
}

int proxima_permutacao(int v[], int n)      /* 1 se avancou, 0 se era a ultima */
{
    int i = n - 2;
    while (i >= 0 && v[i] >= v[i + 1]) i--;       /* 1. pivo */
    if (i < 0) return 0;                          /* sufixo e todo decrescente */
    int j = n - 1;
    while (v[j] <= v[i]) j--;                     /* 2. sucessor do pivo */
    troca(&v[i], &v[j]);                          /* 3. troca */
    inverte(v, i + 1, n - 1);                     /* 4. inverte o sufixo */
    return 1;
}
```

Onde aparece: Knuth [2], seç. 7.2.1.2, Algoritmo L (*lexicographic permutation generation*); Kreher & Stinson [3], seç. 2.4 (sucessor lex). É a mesma rotina do `std::next_permutation` da biblioteca padrão do C++.

Uso e corretude

```
1 int v[] = {1, 2, 3, 4};
2 do {
3     visita(v, 4);
4 } while (proxima_permutacao(v, 4));
```

Comece *ordenado* para obter todas as $n!$ permutações.

Por que é o sucessor

O sufixo à direita do pivô é decrescente: já é o maior arranjo daqueles valores, logo nada muda ali. Aumentar o pivô o mínimo possível e deixar o sufixo *crescente* dá a menor permutação acima da atual.

Análise de custo

Método	Saídas	Tempo total	Espaço extra
Máscara de bits	2^n	$\Theta(n \cdot 2^n)$	$O(1)$
Subconjuntos (backtracking)	2^n	$\Theta(n \cdot 2^n)$	$O(n)$
Permutações (backtracking)	$n!$	$\Theta(n \cdot n!)$	$O(n)$
`proxima_permutacao`	$n!$	$\Theta(n!)$	$O(1)$

Custo amortizado do sucessor

Uma chamada custa $O(n)$ no pior caso, mas o pivô está na última posição em metade dos casos, na penúltima em $1/6$, e assim por diante. O custo médio é $\sum_{k \geq 1} k/k! = e \approx 2{,}7$ comparações: $O(1)$ **amortizado**.

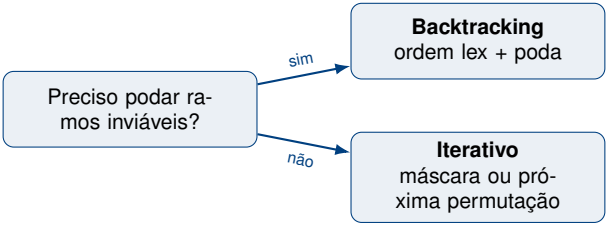
Onde aparece: Knuth [2], seç. 7.2.1.2 (análise do Algoritmo L); Sedgewick [4] (comparação empírica dos geradores).

Recapitulando: sucessor lexicográfico

O que aprendemos

- O sucessor lex se calcula in loco em quatro passos: pivô, troca, inversão.
- Custo $O(n)$ por chamada e $O(1)$ amortizado; espaço extra $O(1)$.
- Permite laço iterativo, sem recursão e sem armazenar permutações.
- O gerador recursivo continua preferível quando há poda a fazer.

Fechamento: qual gerador usar?



- Ordem lex é a mesma nos dois caminhos — muda o custo e a flexibilidade.
- Sempre estime 2^n ou $n!$ *antes* de rodar.

Para praticar

1. Escreva `proximo_subconjunto` que, dado um vetor crescente, produz o próximo subconjunto em ordem lex.
2. Adapte o gerador de subconjuntos para produzir só os de tamanho k, mantendo a ordem lex.
3. Resolva soma de subconjunto com poda e conte quantos nós a poda eliminou.
4. Meça o tempo de `proxima_permutacao` para $n = 8, 9, 10$ e confirme o fator n entre medições consecutivas.

Tutorial guiado: `tutorial/tutorial.ipynb`

Onde cada algoritmo aparece na literatura

Algoritmo da aula	**Referência e local**
Máscara de bits	Knuth [2], seç. 7.2.1.1; Kreher & Stinson [3], seç. 2.2
Subconjuntos por backtracking	Skiena [5], seç. 9.2.1; Ziviani [6], seç. 2.3
Subconjuntos de tamanho k	Kreher & Stinson [3], seç. 2.3; Knuth [2], seç. 7.2.1.3
Poda (soma de subconjunto)	Skiena [5], seç. 9.3; Kreher & Stinson [3], seç. 4.6; Cormen et al. [1], seç. 35.5
Permutações por backtracking	Skiena [5], seç. 9.2.2; Kreher & Stinson [3], seç. 2.4
Permutações por trocas	Sedgewick [4]; Knuth [2], seç. 7.2.1.2 (Algoritmo P)
`proxima_permutacao`	Knuth [2], seç. 7.2.1.2 (Algoritmo L); Kreher & Stinson [3], seç. 2.4
Contagem (2^n e $n!$)	Cormen et al. [1], Apêndice C.1

Onde aparece: O sucessor lexicográfico (Algoritmo L) é atribuído por Knuth a Narayana Pandita, na Índia do século XIV.

Referências I


Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.


Donald E. Knuth.
The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1.
Addison-Wesley, Upper Saddle River, NJ, 2011.


Donald L. Kreher and Douglas R. Stinson.
Combinatorial Algorithms: Generation, Enumeration, and Search.
CRC Press, Boca Raton, FL, 1999.

Referências II


Robert Sedgewick.
Permutation generation methods.
ACM Computing Surveys, 9(2):137–164, 1977.


Steven S. Skiena.
The Algorithm Design Manual.
Springer, Cham, 3 edition, 2020.


Nivio Ziviani.
Projeto de Algoritmos com Implementações em Pascal e C.
Cengage Learning, 3 edition, 2011.
Capítulo 7: algoritmos em grafos.